AI Bootcamp Week 3: Model Invocation, Adversarial Agents, and Bounded Agents

Thursday — Student Exercise Guide

© 2026 Improving

OBJECTIVES

By end of Week 3, the candidate can:

- Explain the DLP automation spectrum and what evidence is required to move up each step
- Explain why the DLP defaults to skills with `disable-model-invocation: true`, and articulate the three graduation criteria for enabling model invocation
- Write invocation test cases (positive trigger, negative trigger, quality) and use them to make a documented enable-or-gap decision on a real skill
- Build an adversarial agent that challenges a skill's output, and explain how it complements (not duplicates) a Python evaluation script
- Run an evaluator-vs-adversarial comparison and use the delta to improve either the EC or the skill
- Compose skills into a bounded agent: a multi-step pipeline run in one IDE session with evaluation and adversarial review at key steps
- Explain context window degradation, why bounded agents hit a ceiling, and what a custom harness is (awareness level; building one is Stage 4 work)
- Describe the IIOS stage framework: where DLP Completion sits (Stage 3, Task) and what Stage 4 (Workflow) requires

© 2026 Improving

TIMING

- **Monday:** 1:00 PM – 5:00 PM (travel morning: 4 hours)
- **Tuesday–Thursday:** 9:00 AM – 5:00 PM with 1-hour lunch (7 hours each)
- **Friday:** 9:00 AM – 12:00 PM (travel afternoon: 3 hours)

© 2026 Improving

THURSDAY (9:00 AM – 5:00 PM): BOUNDED AGENTS AND WHAT COMES AFTER

9:00 – 10:30 AM: WHAT A BOUNDED AGENT IS

Everything you have built so far is single-step: one skill invocation, one model call, one output, human review. A **bounded agent** chains those steps into a multi-step workflow that runs within the IDE's built-in harness. You invoke a skill, review the output, invoke the next skill, review that output, and so on. The context window carries artifacts between steps: the stories are in context when you invoke `plan-story`, and the plan is in context when you invoke `implement-story`.

This is not a new tool. It is not a Python program or API calls. It is you, in your IDE, running skills you already built, in sequence, with your evaluation script and adversarial agent applied at key steps.

What a bounded agent can do:

- Run the full SDLC pipeline: `create-epics-stories` → `plan-story` → `implement-story` → `review-implementation`, in one session
- Run the evaluation script after each step (you invoke it manually, but it runs)
- Run the adversarial agent after key steps
- Produce a final artifact that includes all step outputs, evaluation scores, and adversarial findings

What a bounded agent cannot do:

- Give each step its own clean context. The context window accumulates, and output quality degrades at later steps.
- Evaluate automatically. You run the evaluation script manually at each step.
- Handle failures programmatically. If a step fails evaluation, you decide what to do, not the harness.
- Run unattended. You are in the loop at every step.

A bounded agent is a manually orchestrated chain with checkpoints, not automation. Treating it as automation is the failure mode. The human at each checkpoint is the governance.

10:30 AM – 12:00 PM: BUILD A BOUNDED AGENT

Run the full pipeline in a single IDE session, starting from a fresh session and a single story or feature request:

1. Invoke `create-epics-stories` (or start from `create-prd` if you have the full Week 2 chain working). Review the output.
2. Run the evaluation script against the output. Note the EC scores.
3. Invoke `plan-story`. Review the plan. Evaluate it.
4. Invoke `implement-story`. Review the code.
5. Run your adversarial agent against the code output. Note findings.
6. Invoke `review-implementation`. Review the review.

© 2026 Improving

7. Save the full session output as your bounded agent artifact.

Done when: you have run the chain end to end in one session, with evaluation and adversarial review at key steps, and you can point to where output quality changed at later steps.

12:00 – 1:00 PM: LUNCH

1:00 – 2:00 PM: CONTEXT WINDOW DISCIPLINE (FEEL IT FIRST, THEN NAME IT)

You just ran the full pipeline. You likely noticed: later steps produced output that was less sharp than earlier ones. That is not a model problem. It is a context window problem.

The degradation pattern you just experienced:

- Step 1 had a clean context window: good output
- Step 2 had Step 1's output in context: still good, slightly constrained
- Step 3 had Steps 1 and 2 in context: noticeable quality difference in complex cases
- Step 4 had everything in context: often where quality falls off in complex inputs

The model's attention spreads across the entire accumulated context. The IDE harness does not manage this for you; the window just grows.

What you can do about it within the IDE:

- Keep each step's output compact. Verbose output crowds the next step.
- Evaluate at each step. Catch degradation early, before it compounds.
- If a step degrades, start a fresh session with only the prior artifact pasted in: a manual clean-context workaround.
- Accept that the bounded agent has a ceiling. For long pipelines, the window fills and quality degrades.

2:00 – 3:00 PM: THE CUSTOM HARNESS (AWARENESS ONLY)

The solution to the bounded agent's ceiling is a **custom harness**: a Python program that orchestrates multiple model calls with clean context per step, passes artifacts between steps programmatically (reading and writing the same YAML files the skills use), evaluates each step automatically with the same evaluation scripts, runs adversarial agents at key points, and produces a final artifact with scores at every step.

The key insight: **the custom harness does not replace the skills. It composes them.** Everything you have built (skills, EC, evaluation scripts, adversarial agents) becomes the building blocks. The harness is how you put them together into a production workflow.

You will not build one in this program. Building a custom harness is the work of a future Stage 4 Deep Learning Program. What you need as an Embedded Engineer is awareness: what it is, what problem it solves, and when a client team is ready for it.

3:00 – 5:00 PM: THE IIOS STAGE FRAMEWORK

The **Improving Intelligence Operating System (IIOS)** defines stages of AI maturity:

© 2026 Improving

Stage	Name	What It Means	Median Effort
2	Off the Shelf	Using AI tools as-is. Producing with AI but not measuring impact.	n/a
3	Task	Defined tasks with real context. Deliberate RTCC skills, measured improvement, evidence-based iteration, adversarial challenge, bounded agents.	18h
4	Workflow	Chained multi-step workflows with automated harnesses, guardrails, human punch-out points, end-to-end success rates, audit trails.	172h (~10x Stage 3)

DLP Completion is Stage 3. Everything in this bootcamp and the 12-week DLP targets it: personal skills, deliberate invocation decisions, measured iteration, adversarial challenge, bounded agents with a human at every checkpoint.

The Stage 3 ceiling: Stage 3 is personal. Each person's skills reflect their own standards. Review is a human, one piece at a time. To compound, the team must reconcile personal agents into shared workflows and move checking out of human heads and into systems. That is Stage 4.

What Stage 4 requires: team coordination with IT oversight, roughly 10x the effort of Stage 3, and solid Stage 3 foundations. A Stage 4 workflow built on unreliable Stage 3 skills is not faster. It is a slop cannon: errors compound through every connected step. (At 95% per-step accuracy, a 20-step pipeline completes correctly only about 36% of the time.)

Why this matters to you as an Embedded Engineer: client teams will ask "can we automate more of this?" by Week 3 of an engagement. You now have the honest answer: what the prerequisites are (evaluated, iterated, adversarially challenged skills), what the failure modes are at each step of the spectrum, and what the next stage actually costs. You can describe the trajectory without overpromising.

© 2026 Improving